AMD DEVELOPER HACKATHON ACT III · TRACK 1: INTELLIGENT INDUSTRY

LineGuard

Catch machine failures a day earlier.
Fix them in a planned stop.

Predictive maintenance copilot · AI running on AMD Instinct GPUs

[Team name] · [Team members]

THE PROBLEM

Fixed alarms fire too late to plan a repair

~10 h

median warning a fixed alarm limit gave before the trip, across 149 developing faults

24 h

lead time for a pump bearing kit, in our example plant's maintenance manual

11 h

downtime when that pump then failed unplanned, from the example work-order history

The data is already there. The warning just arrives too late to order the part and plan the stop, so the repair happens **during production**.

WHO IT IS FOR

Two people decide whether a fault becomes a breakdown

Shift operator

- What is wrong with this machine?
- How urgent is it?
- What do I check first?

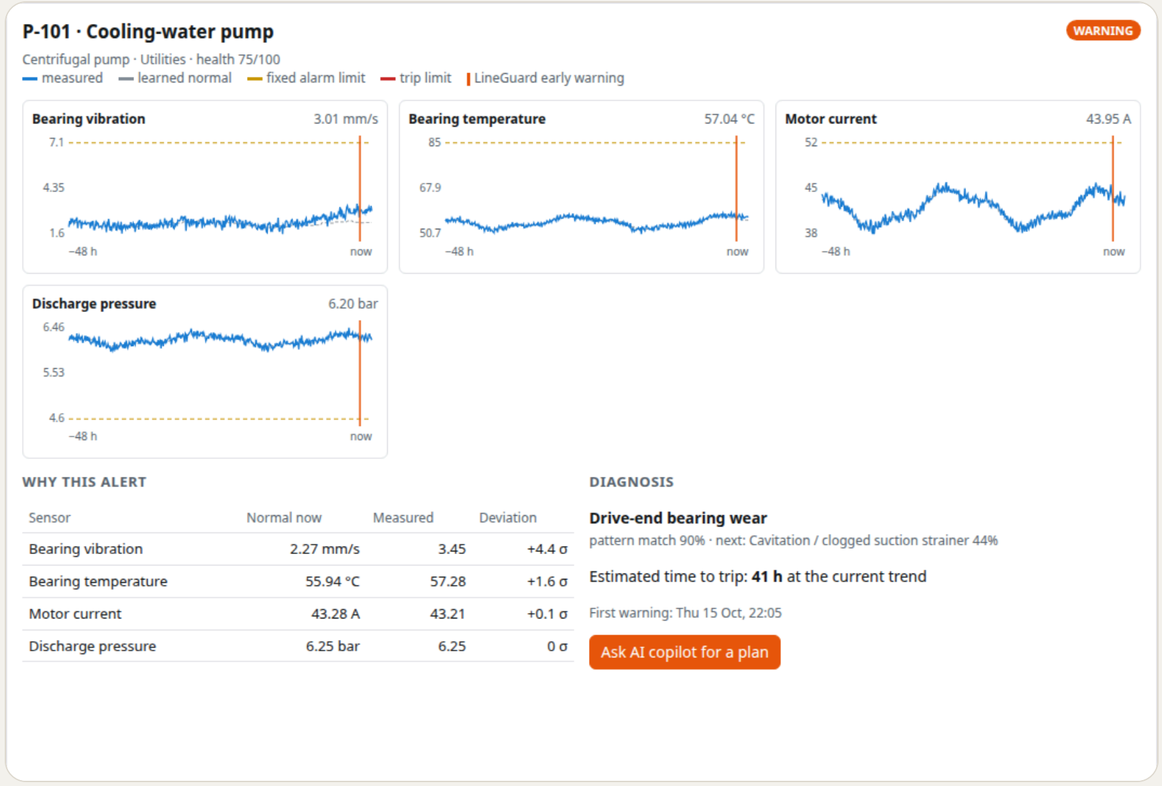
Maintenance planner

- Which part do I order?
- How long is the job?
- Which planned stop can we use?

LineGuard answers both on one screen, in plain language, with sources.

1 · SEE IT EARLY

A drift shows up a day before any alarm



Learns each machine’s normal

For every sensor and every hour of the shift. The grey dotted line is the machine's own normal.

Alerts on persistent drift

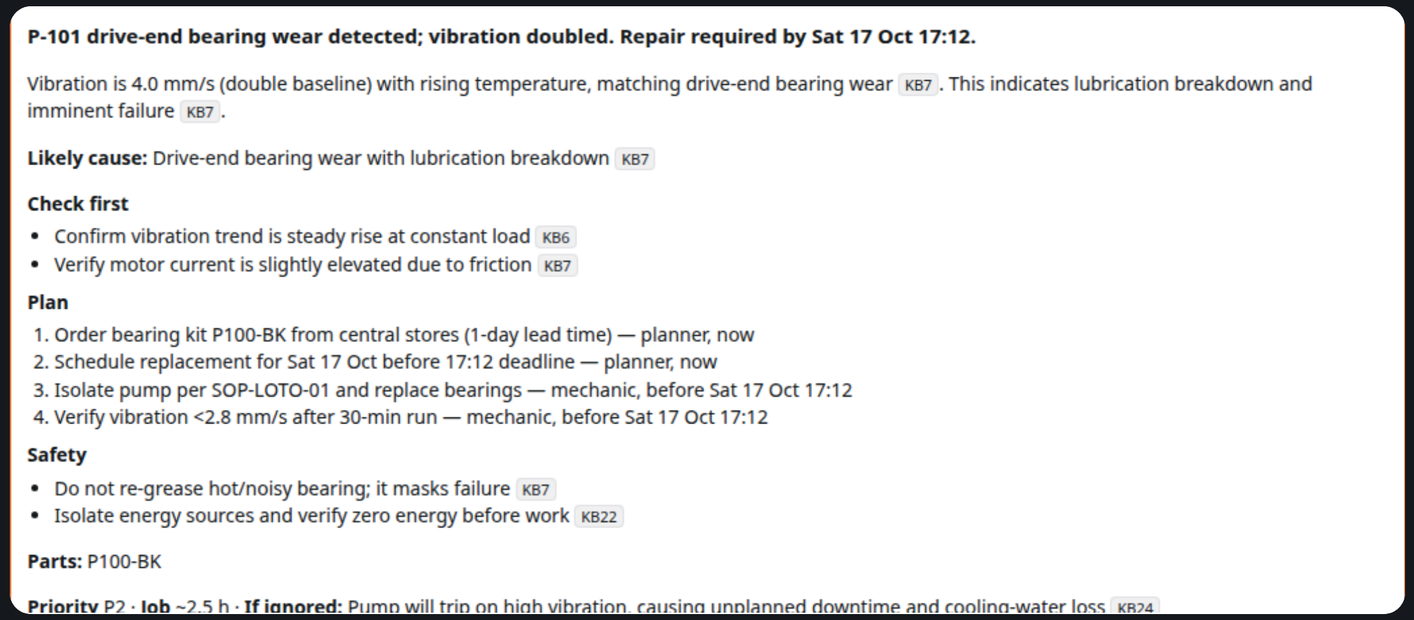
A deviation must hold for 15 minutes, so one-off sensor glitches never raise an alert.

Predicts when it will trip

Matches the fault (bearing wear, 97%) and fits the accelerating trend to the trip limit: "41 h to trip". Actual: 40 h.

2 · KNOW WHAT TO DO

The AI on AMD writes the repair plan



P-101 drive-end bearing wear detected; vibration doubled. Repair required by Sat 17 Oct 17:12.

Vibration is 4.0 mm/s (double baseline) with rising temperature, matching drive-end bearing wear KB7. This indicates lubrication breakdown and imminent failure KB7.

Likely cause: Drive-end bearing wear with lubrication breakdown KB7

Check first

- Confirm vibration trend is steady rise at constant load KB6
- Verify motor current is slightly elevated due to friction KB7

Plan

1. Order bearing kit P100-BK from central stores (1-day lead time) — planner, now
2. Schedule replacement for Sat 17 Oct before 17:12 deadline — planner, now
3. Isolate pump per SOP-LOTO-01 and replace bearings — mechanic, before Sat 17 Oct 17:12
4. Verify vibration <2.8 mm/s after 30-min run — mechanic, before Sat 17 Oct 17:12

Safety

- Do not re-grease hot/noisy bearing; it masks failure KB7
- Isolate energy sources and verify zero energy before work KB22

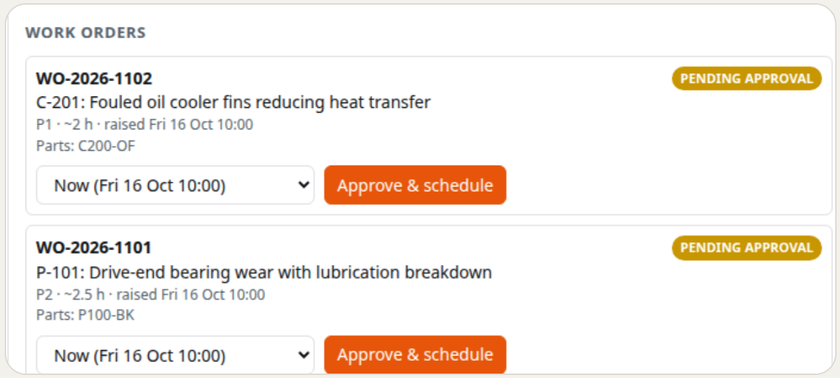
Parts: P100-BK

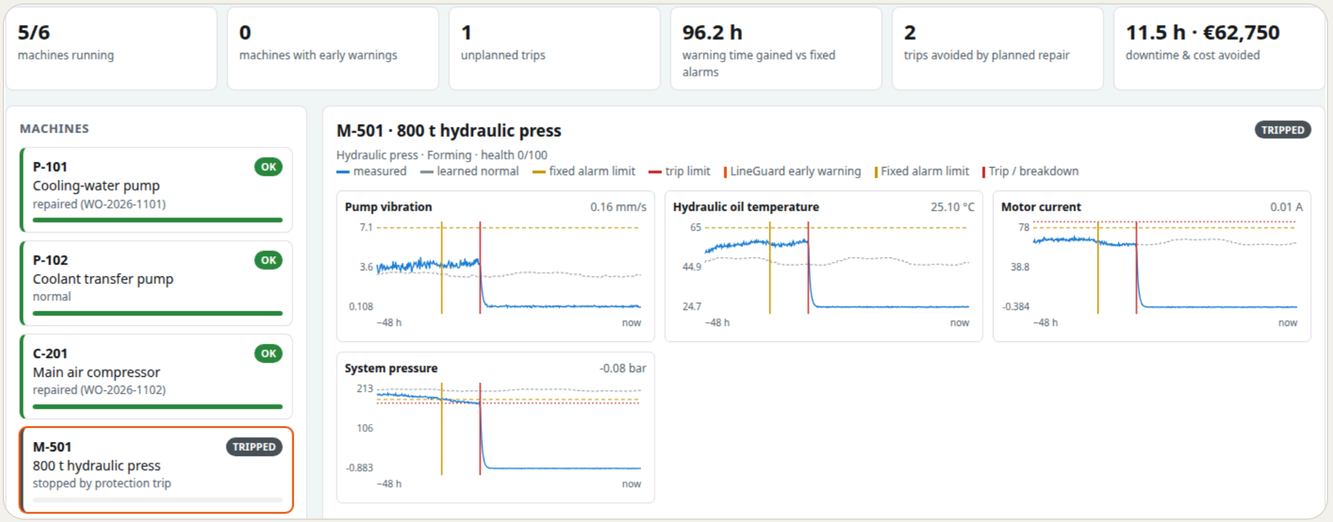
Priority P2 · **Job** ~2.5 h · **If ignored:** Pump will trip on high vibration, causing unplanned downtime and cooling-water loss KB24

- Cites the plant's own manuals and incident history
- Checks, part number, job time, safety steps and priority
- Qwen3.8-Flash-Next on AMD Token Factory

Detection, time-to-trip and costs come from code. The model explains and plans; it never makes up the numbers.

Approve the plan, repair in a planned stop





Same week, two outcomes: P-101 and C-201 were **repaired in planned stops**; the press nobody acted on **broke down**. Result: **€62,750** and 11.5 h of downtime avoided.

HOW IT WORKS

From sensor drift to scheduled repair

1

Sensors

Vibration, temperature, current and pressure, every minute.

2

Detector

Flags persistent drift, names the fault, estimates time to trip. Plain, testable code.

3 · on AMD

AI copilot

Finds the right manual sections and past work orders, writes a cited plan.

4

Work order

A planner approves a slot. The repair happens in a planned stop.

Numbers come from code. The language model only explains and plans.

RUNS ON AMD

The copilot’s model runs on AMD Instinct GPUs

What	Measured in this project
Endpoint	AMD Token Factory, OpenAI-compatible API
Model	Qwen3.8-Flash-Next, open weights
Plan generation	18–30 s per alert, about 20–25 tokens/s
Benchmark	9 of 9 plans were valid and cited (Qwen3.8 and DeepSeek-V4)
In the demo	Instant: the plan starts in the background when an alert opens



Same code, plant’s own GPU

Point it at a vLLM server on an AMD Instinct MI300X, and sensor data and manuals can stay inside the plant.

RESULTS

Measured on 50 unseen fault scenarios

33 h

median warning before the trip (fixed alarms: 10 h)

100%

of 149 developing faults warned before the trip

0

false alerts on healthy machines

€18.7k

downtime cost avoided per fault, and 3.1 h less downtime

Time-to-trip prediction one day ahead: **within about ±15%** of the real trip time (median 0.96×).

Plant data is simulated: physically motivated fault patterns, shift cycles, noise and sensor glitches, with exact ground truth. Real data is noisier, so expect some false alerts and tuning. Costs use the editable assumptions in lineguard/impact.py. Reproduce with: python -m scripts.evaluate --runs 50

TRUST AND SAFETY

Built so operators can trust it

- Every claim cites a source. Citations the model can't back up are struck out.
- Lockout/tagout is added to every hands-on plan, always.
- Detection, time-to-trip and costs are computed by code, not by the model.
- A person approves every work order and picks the slot.
- If the AI is unreachable, a clearly labelled fallback keeps the screen working.
- Tests cover detection, causality, diagnosis, the AI path and the work-order loop.

BUSINESS VALUE

Value today, and the path to a real plant

Why a plant pays

- About €18.7k avoided per caught fault (example cost model)
- Uses the sensors plants already have
- Pricing: [€__ per monitored machine per month]
- Payback: one caught fault a year covers [__] machines

Next steps

1. Connect to the plant historian or OPC UA
2. Pilot on one real line with its failure history
3. Self-host the model on an AMD Instinct MI300X
4. Learn new failure modes from the plant's own work orders

LINEGUARD

Catch it a day earlier. Fix it in a planned stop.

Code: [github.com/your-team/lineguard]

Live demo: [your demo URL]

[Team name] · AMD Developer Hackathon Act III · Track 1